Retail Sales Analysis - SQL Queries with Output

Each section shows the SQL query, a screenshot of the result and the insight. Data: 8,939 cleaned order lines normalised into customers, products, orders and order_lines. Engine: SQLite.

DQ1 - Row counts per table

```
SELECT 'customers' AS tbl, COUNT(*) AS n FROM customers UNION ALL SELECT 'products',COUNT(*) FROM products
UNION ALL SELECT 'orders',COUNT(*) FROM orders UNION ALL SELECT 'order_lines',COUNT(*) FROM order_lines;
```

Output (screenshot):

tbl	n
customers	832
products	65
orders	4,295
order_lines	8,939

Insight: Row counts of the four normalised tables - all populated.

DQ2 - Staging vs fact reconciliation (totals must match exactly)

```
SELECT ROUND((SELECT SUM(sales) FROM stg_sales),2) AS staging_sales, ROUND((SELECT SUM(sales) FROM order_lines),2) AS
    fact_sales,
       ROUND((SELECT SUM(profit) FROM stg_sales),2) AS staging_profit, ROUND((SELECT SUM(profit) FROM order_lines),2)
    AS fact_profit;
```

Output (screenshot):

staging_sales	fact_sales	staging_profit	fact_profit
53,583,538	53,583,538	7,417,634	7,417,634

Insight: Staging and fact totals reconcile to the rupee: nothing was lost or double-counted when loading.

DQ3 - NULL / invalid value checks

```
SELECT SUM(CASE WHEN sales IS NULL THEN 1 ELSE 0 END)  AS null_sales,
       SUM(CASE WHEN sales <= 0 THEN 1 ELSE 0 END)      AS non_positive_sales,
       SUM(CASE WHEN discount < 0 OR discount > 1 THEN 1 ELSE 0 END) AS bad_discount,
       SUM(CASE WHEN quantity <= 0 THEN 1 ELSE 0 END)   AS bad_quantity
FROM order_lines;
```

Output (screenshot):

null_sales	non_positive_sales	bad_discount	bad_quantity
0	0	0	0

Insight: No NULL, non-positive sales, bad discounts or bad quantities.

DQ4 - Ship date earlier than order date

```
SELECT COUNT(*) AS bad_ship_dates FROM orders WHERE ship_date < order_date;
```

Output (screenshot):

bad_ship_dates
0

Insight: No order ships before it is placed.

DQ5 - Orphan check: lines without a matching order / product

```
SELECT COUNT(*) AS orphan_lines FROM order_lines l
LEFT JOIN orders o ON o.order_id = l.order_id LEFT JOIN products p ON p.product_id = l.product_id
WHERE o.order_id IS NULL OR p.product_id IS NULL;
```

Output (screenshot):

orphan_lines
0

Insight: No orphan lines: every line has a valid order and product.

DQ6 - Duplicate business keys in staging (should be 0)

```
SELECT COUNT(*) AS duplicate_keys FROM (SELECT order_id, product_id FROM stg_sales GROUP BY 1,2 HAVING COUNT(*) > 1);
```

Output (screenshot):

duplicate_keys
0

Insight: No duplicate order-line keys remain after cleaning.

DQ7 - Date coverage

```
SELECT MIN(order_date) AS first_order, MAX(order_date) AS last_order, COUNT(DISTINCT substr(order_date,1,7)) AS
    months_covered FROM orders;
```

Output (screenshot):

first_order	last_order	months_covered
2023-01-01	2025-12-31	36

Insight: 36 full months of data (Jan 2023 - Dec 2025).

Q1 - Executive KPI summary

```
SELECT ROUND(SUM(l.sales),0)                        AS total_sales,
       ROUND(SUM(l.profit),0)                       AS total_profit,
       ROUND(100.0*SUM(l.profit)/SUM(l.sales),2)    AS profit_margin_pct,
       COUNT(DISTINCT l.order_id)                   AS orders,
       COUNT(DISTINCT o.customer_id)                AS customers,
       SUM(l.quantity)                              AS units_sold,
       ROUND(SUM(l.sales)/COUNT(DISTINCT l.order_id),0) AS avg_order_value
FROM order_lines l JOIN orders o ON o.order_id = l.order_id;
```

Output (screenshot):

total_sales	total_profit	profit_margin_pct	orders	customers	units_sold	avg_order_value
53,583,538	7,417,634	13.84	4,295	832	22,747	12,476

Insight: INR 53.6 M sales, INR 7.4 M profit (13.8% margin), 4,295 orders from 832 customers; average order INR 12,476.

Q2 - Year-over-year growth (LAG window function)

```
WITH yearly AS (
  SELECT substr(o.order_date,1,4) AS yr, SUM(l.sales) AS sales, SUM(l.profit) AS profit
  FROM order_lines l JOIN orders o ON o.order_id=l.order_id GROUP BY 1)
SELECT yr, ROUND(sales,0) AS sales, ROUND(profit,0) AS profit,
       ROUND(100.0*profit/sales,2) AS margin_pct,
       ROUND(100.0*(sales - LAG(sales) OVER (ORDER BY yr)) / LAG(sales) OVER (ORDER BY yr),2) AS sales_yoy_pct,
       ROUND(100.0*(profit - LAG(profit) OVER (ORDER BY yr)) / LAG(profit) OVER (ORDER BY yr),2) AS profit_yoy_pct
FROM yearly ORDER BY yr;
```

Output (screenshot):

yr	sales	profit	margin_pct	sales_yoy_pct	profit_yoy_pct
2023	14,367,369	1,949,391	13.57		
2024	17,981,033	2,582,543	14.36	25.15	32.48
2025	21,235,136	2,885,700	13.59	18.10	11.74

Insight: Sales grew 25% (2024) and 18% (2025) but margin stayed flat at ~14% - growth is not improving profitability.

Q3 - Monthly sales, MoM growth and 3-month moving average (window frame)

```
WITH monthly AS (
  SELECT substr(o.order_date,1,7) AS ym, SUM(l.sales) AS sales
  FROM order_lines l JOIN orders o ON o.order_id=l.order_id GROUP BY 1)
SELECT ym, ROUND(sales,0) AS sales,
       ROUND(100.0*(sales-LAG(sales) OVER (ORDER BY ym))/LAG(sales) OVER (ORDER BY ym),1) AS mom_pct,
       ROUND(AVG(sales) OVER (ORDER BY ym ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),0) AS moving_avg_3m
FROM monthly ORDER BY ym LIMIT 12;
```

Output (screenshot):

ym	sales	mom_pct	moving_avg_3m
2023-01	797,215		797,215
2023-02	684,493	-14.10	740,854
2023-03	1,170,223	71	883,977
2023-04	865,470	-26	906,729
2023-05	1,015,054	17.30	1,016,916
2023-06	618,417	-39.10	832,980
2023-07	1,435,915	132	1,023,129
2023-08	1,344,110	-6.40	1,132,814
2023-09	1,330,635	-1	1,370,220
2023-10	2,086,298	56.80	1,587,015
2023-11	1,321,064	-36.70	1,579,333
2023-12	1,698,475	28.60	1,701,946

Insight: Monthly sales are volatile; the 3-month moving average smooths the noise and shows the upward trend.

Q4 - Category -> sub-category profitability with ranking and loss flag

```
SELECT p.category, p.sub_category, ROUND(SUM(l.sales),0) AS sales, ROUND(SUM(l.profit),0) AS profit,
       ROUND(100.0*SUM(l.profit)/SUM(l.sales),2) AS margin_pct,
       ROUND(AVG(l.discount)*100,1) AS avg_discount_pct,
       RANK() OVER (ORDER BY SUM(l.profit) DESC) AS profit_rank,
       CASE WHEN SUM(l.profit) < 0 THEN 'LOSS-MAKING' ELSE 'profitable' END AS status
FROM order_lines l JOIN products p ON p.product_id=l.product_id
GROUP BY p.category, p.sub_category ORDER BY profit DESC;
```

Output (screenshot):

category	sub_category	sales	profit	margin_pct	avg_discount_pct	profit_rank	status
Technology	Copiers	7,127,571	1,757,144	24.65	6.70	1	profitable
Office Supplies	Storage	7,781,841	1,406,963	18.08	12	2	profitable
Furniture	Chairs	8,375,257	1,290,564	15.41	11.20	3	profitable
Furniture	Bookcases	7,529,323	791,396	10.51	12.50	4	profitable
Technology	Phones	6,936,844	689,047	9.93	12.90	5	profitable
Office Supplies	Paper	1,603,046	668,973	41.73	6.20	6	profitable
Office Supplies	Art	1,550,327	524,840	33.85	6.30	7	profitable
Furniture	Furnishings	919,945	288,531	31.36	6.60	8	profitable
Office Supplies	Binders	866,395	208,138	24.02	21.20	9	profitable
Office Supplies	Labels	529,887	202,855	38.28	6	10	profitable
Technology	Accessories	179,665	52,949	29.47	8.70	11	profitable
Technology	Machines	3,480,089	-108,067	-3.11	22.40	12	LOSS-MAKING
Furniture	Tables	6,703,348	-355,697	-5.31	21.50	13	LOSS-MAKING

Insight: Copiers, Storage and Chairs earn the most profit; Tables and Machines (bottom of the list) are loss-making.

Q5 - Region performance (sales, margin, discount behaviour)

```
SELECT c.region, ROUND(SUM(l.sales),0) AS sales, ROUND(SUM(l.profit),0) AS profit,
       ROUND(100.0*SUM(l.profit)/SUM(l.sales),2) AS margin_pct,
       ROUND(100.0*AVG(l.discount),1) AS avg_discount_pct,
       ROUND(100.0*SUM(CASE WHEN l.profit<0 THEN 1 ELSE 0 END)/COUNT(*),1) AS pct_lines_loss_making
FROM order_lines l JOIN orders o ON o.order_id=l.order_id JOIN customers c ON c.customer_id=o.customer_id
GROUP BY c.region ORDER BY margin_pct DESC;
```

Output (screenshot):

region	sales	profit	margin_pct	avg_discount_pct	pct_lines_loss_making
South	16,971,575	2,537,605	14.95	10.10	8
East	9,156,893	1,296,894	14.16	9.80	7.20
West	11,529,617	1,577,024	13.68	10.10	8.50
North	10,525,532	1,357,842	12.90	10.20	7.80
Central	5,399,921	648,270	12.01	13.20	12.20

Insight: Central has the lowest margin (12.0%), the deepest average discount (13.2%) and the most loss-making lines (12.2%).

Q6 - Discount band analysis - the core "why are we losing money" query

```
SELECT CASE WHEN discount=0 THEN '0. No discount' WHEN discount<=0.10 THEN '1. 1-10%'
            WHEN discount<=0.20 THEN '2. 11-20%'  WHEN discount<=0.30 THEN '3. 21-30%' ELSE '4. Above 30%' END AS
    discount_band,
       COUNT(*) AS order_lines, ROUND(SUM(sales),0) AS sales, ROUND(SUM(profit),0) AS profit,
       ROUND(100.0*SUM(profit)/SUM(sales),2) AS margin_pct,
       ROUND(100.0*SUM(CASE WHEN profit<0 THEN 1 ELSE 0 END)/COUNT(*),1) AS pct_lines_loss
FROM order_lines GROUP BY 1 ORDER BY 1;
```

Output (screenshot):

discount_band	order_lines	sales	profit	margin_pct	pct_lines_loss
0. No discount	3,725	21,101,165	5,732,963	27.17	0
1. 1-10%	2,207	12,723,249	2,478,915	19.48	0
2. 11-20%	1,723	11,248,763	1,050,503	9.34	1.30
3. 21-30%	701	4,799,626	-468,111	-9.75	37.90
4. Above 30%	583	3,710,734	-1,376,636	-37.10	79.20

Insight: The key finding: margin falls from 27% (no discount) to 9% (11-20%) and turns negative above 20% (-10%, then -37%). 79% of lines with >30% discount lose money.

Q7 - Ten worst products by profit (candidates for re-pricing or delisting)

```
SELECT p.product_id, p.product_name, p.sub_category, SUM(l.quantity) AS units,
       ROUND(SUM(l.sales),0) AS sales, ROUND(SUM(l.profit),0) AS profit,
       ROUND(100.0*AVG(l.discount),1) AS avg_discount_pct
FROM order_lines l JOIN products p ON p.product_id=l.product_id
GROUP BY p.product_id, p.product_name, p.sub_category ORDER BY profit ASC LIMIT 10;
```

Output (screenshot):

product_id	product_name	sub_category	units	sales	profit	avg_discount_pct
PRD-0010	Ultra Table E10	Tables	134	1,928,604	-220,175	26.90
PRD-0062	Prime Machine B62	Machines	27	1,087,349	-103,838	24.10
PRD-0009	Eco Table D9	Tables	91	1,644,682	-89,420	22.60
PRD-0006	Elite Table A6	Tables	102	724,180	-62,753	23.20
PRD-0065	Deluxe Machine E65	Machines	36	466,964	-53,854	29.20
PRD-0064	Basic Machine D64	Machines	40	773,503	-43,517	21.50
PRD-0007	Compact Table B7	Tables	145	1,218,415	-10,750	18.90
PRD-0049	Eco Phone D49	Phones	27	469,586	-10,677	21.10
PRD-0055	Deluxe Accessory E55	Accessories	44	21,271	5,408	10.90
PRD-0052	Prime Accessory B52	Accessories	9	18,367	5,817	5

Insight: Seven of the ten worst products are Tables or Machines, all discounted 19-29% on average - candidates for re-pricing or delisting.

Q8 - Pareto: how concentrated is profit? Top 10 customers + cumulative share

```
WITH cust AS (
  SELECT o.customer_id, SUM(l.profit) AS profit
  FROM order_lines l JOIN orders o ON o.order_id=l.order_id GROUP BY 1),
ranked AS (SELECT customer_id, profit, SUM(profit) OVER () AS total FROM cust)
SELECT customer_id, ROUND(profit,0) AS profit,
       ROUND(100.0*SUM(profit) OVER (ORDER BY profit DESC)/total,1) AS cumulative_profit_pct
FROM ranked ORDER BY profit DESC LIMIT 10;
```

Output (screenshot):

customer_id	profit	cumulative_profit_pct
CUST-0539	252,734	3.40
CUST-0107	206,499	6.20
CUST-0182	193,190	8.80
CUST-0178	191,006	11.40
CUST-0090	153,200	13.40
CUST-0481	143,378	15.40
CUST-0152	124,594	17
CUST-0661	118,413	18.60
CUST-0067	101,725	20
CUST-0027	99,830	21.40

Insight: Profit is spread out: the top 10 customers contribute roughly a fifth of total profit.

Q9 - RFM customer segmentation (NTILE) - who to retain / win back

```
WITH base AS (
  SELECT o.customer_id, MAX(o.order_date) AS last_order, COUNT(DISTINCT o.order_id) AS frequency, SUM(l.sales) AS
    monetary
  FROM orders o JOIN order_lines l ON l.order_id=o.order_id GROUP BY 1),
scored AS (
  SELECT customer_id, frequency, monetary,
         CAST(julianday((SELECT MAX(order_date) FROM orders)) - julianday(last_order) AS INT) AS recency_days,
         NTILE(5) OVER (ORDER BY last_order)  AS r,   -- 5 = most recent
         NTILE(5) OVER (ORDER BY frequency)   AS f,
         NTILE(5) OVER (ORDER BY monetary)    AS m
  FROM base)
SELECT CASE WHEN r>=4 AND f>=4 AND m>=4 THEN 'Champions'
            WHEN r>=3 AND f>=3 THEN 'Loyal'
            WHEN r<=2 AND (f>=4 OR m>=4) THEN 'At risk - win back'
            WHEN r<=2 THEN 'Lost / low value' ELSE 'Potential' END AS rfm_segment,
       COUNT(*) AS customers, ROUND(AVG(recency_days),0) AS avg_recency_days,
       ROUND(AVG(frequency),1) AS avg_orders, ROUND(SUM(monetary),0) AS total_sales
FROM scored GROUP BY 1 ORDER BY total_sales DESC;
```

Output (screenshot):

rfm_segment	customers	avg_recency_days	avg_orders	total_sales
Champions	172	33	13.20	33,593,064
Loyal	203	96	4.90	8,957,647
At risk - win back	102	408	4	6,542,931
Lost / low value	232	499	1.90	2,410,077
Potential	123	90	1.60	2,079,819

Insight: 172 'Champions' generate INR 33.6 M (63%) of sales; 102 valuable customers are 'at risk' (about 408 days since their last order) - a win-back target.

Q10 - Delivery performance by region and ship mode

```
SELECT c.region, o.ship_mode, COUNT(*) AS orders,
       ROUND(AVG(julianday(o.ship_date)-julianday(o.order_date)),2) AS avg_delivery_days,
       ROUND(100.0*SUM(CASE WHEN julianday(o.ship_date)-julianday(o.order_date) > 7 THEN 1 ELSE 0 END)/COUNT(*),1) AS
    pct_over_7_days
FROM orders o JOIN customers c ON c.customer_id=o.customer_id
GROUP BY c.region, o.ship_mode HAVING o.ship_mode='Standard Class' ORDER BY avg_delivery_days DESC;
-- MySQL: DATEDIFF(ship_date, order_date)   PostgreSQL: (ship_date - order_date)
```

Output (screenshot):

region	ship_mode	orders	avg_delivery_days	pct_over_7_days
Central	Standard Class	264	7.02	41.30
East	Standard Class	429	6.21	23.30
South	Standard Class	785	6.09	23.10
West	Standard Class	572	6.02	18
North	Standard Class	488	5.91	19.70

Insight: Standard Class delivery in Central takes 7.0 days and 41% of orders exceed 7 days, versus 18-23% elsewhere.

Q11 - Top 3 sub-categories by profit in each region (ROW_NUMBER inside a partition)

```
WITH x AS (
  SELECT c.region, p.sub_category, SUM(l.profit) AS profit,
         ROW_NUMBER() OVER (PARTITION BY c.region ORDER BY SUM(l.profit) DESC) AS rn
  FROM order_lines l JOIN orders o ON o.order_id=l.order_id JOIN customers c ON c.customer_id=o.customer_id
  JOIN products p ON p.product_id=l.product_id GROUP BY c.region, p.sub_category)
SELECT region, rn AS rank, sub_category, ROUND(profit,0) AS profit FROM x WHERE rn<=3 ORDER BY region, rn;
```

Output (screenshot):

region	rank	sub_category	profit
Central	1	Copiers	294,864
Central	2	Storage	120,915
Central	3	Paper	74,842
East	1	Storage	258,405
East	2	Copiers	255,097
East	3	Chairs	187,674
North	1	Storage	271,792
North	2	Chairs	253,130
North	3	Bookcases	177,427
South	1	Copiers	780,005
South	2	Storage	479,918
South	3	Chairs	433,494
West	1	Chairs	380,060
West	2	Storage	275,934
West	3	Copiers	261,076

Insight: Copiers, Storage, Chairs and Paper dominate profit in every region - a useful guide for regional stocking.

Q12 - Seasonality index: each month vs the average month (100 = average)

```
WITH m AS (
  SELECT CAST(substr(o.order_date,6,2) AS INT) AS month_no, SUM(l.sales) AS sales
  FROM order_lines l JOIN orders o ON o.order_id=l.order_id GROUP BY 1)
SELECT month_no, ROUND(sales,0) AS sales, ROUND(100.0*sales/AVG(sales) OVER (),0) AS seasonality_index FROM m ORDER BY
    month_no;
```

Output (screenshot):

month_no	sales	seasonality_index
1	3,588,734	80
2	3,293,379	74
3	4,156,796	93
4	3,961,173	89
5	4,687,089	105
6	3,224,139	72
7	3,897,904	87
8	4,818,928	108
9	5,003,710	112
10	5,397,774	121
11	6,114,530	137
12	5,439,381	122

Insight: A festive-season peak appears in Oct-Dec (index above 120); Jan, Feb and Jun are the weakest months.

Q13 - Segment x category margin matrix

```
SELECT c.segment, p.category, ROUND(SUM(l.sales),0) AS sales, ROUND(100.0*SUM(l.profit)/SUM(l.sales),2) AS margin_pct
FROM order_lines l JOIN orders o ON o.order_id=l.order_id JOIN customers c ON c.customer_id=o.customer_id
JOIN products p ON p.product_id=l.product_id GROUP BY c.segment, p.category ORDER BY c.segment, margin_pct DESC;
```

Output (screenshot):

segment	category	sales	margin_pct
Consumer	Office Supplies	6,612,980	24.83
Consumer	Technology	10,544,499	14
Consumer	Furniture	11,919,175	8.90
Corporate	Office Supplies	3,669,110	23.96
Corporate	Technology	5,028,598	10.46
Corporate	Furniture	8,082,973	9.17
Home Office	Office Supplies	2,049,406	23.94
Home Office	Technology	2,151,073	18.08
Home Office	Furniture	3,525,724	6.06

Insight: Furniture earns under 10% margin in every segment, while Office Supplies earn ~24% - product mix matters more than customer segment.

Q14 - Repeat-purchase rate

```
WITH n AS (SELECT customer_id, COUNT(*) AS orders FROM orders GROUP BY 1)
SELECT COUNT(*) AS customers, SUM(CASE WHEN orders>1 THEN 1 ELSE 0 END) AS repeat_customers,
       ROUND(100.0*SUM(CASE WHEN orders>1 THEN 1 ELSE 0 END)/COUNT(*),1) AS repeat_rate_pct, ROUND(AVG(orders),2) AS
    avg_orders_per_customer FROM n;
```

Output (screenshot):

customers	repeat_customers	repeat_rate_pct	avg_orders_per_customer
832	674	81	5.16

Insight: 81% of customers buy more than once (5.2 orders per customer on average): the business is repeat-driven.

Q15 - Discount leakage: profit destroyed by deep-discount (>20%) lines, by sub-category

```
SELECT p.sub_category, COUNT(*) AS deep_discount_lines, ROUND(SUM(l.profit),0) AS profit_on_those_lines,
       ROUND(100.0*SUM(l.profit)/(SELECT SUM(profit) FROM order_lines WHERE profit<0),1) AS share_of_total_losses_pct
FROM order_lines l JOIN products p ON p.product_id=l.product_id
WHERE l.discount > 0.20 AND l.profit < 0 GROUP BY p.sub_category ORDER BY profit_on_those_lines ASC LIMIT 6;
```

Output (screenshot):

sub_category	deep_discount_lines	profit_on_those_lines	share_of_total_losses_pct
Tables	109	-804,391	42.40
Machines	20	-353,028	18.60
Phones	26	-220,907	11.60
Bookcases	50	-214,659	11.30
Chairs	43	-143,650	7.60
Storage	261	-114,986	6.10

Insight: Tables alone account for 42% of all losses from deep-discount lines, followed by Machines (19%) - capping discounts on these two sub-categories fixes most of the leak.